Pollard-Rho 大整数分解

用于分解 64 位无符号整数。代码沿用火车头中的 `u64`、`u128`、`solve()` 和多测 `main()` 结构，可直接提交。

适用范围

- `is_prime(n)`：确定性 Miller-Rabin，对整个 `u64` 范围正确。
- `max_prime_factor(n)`：返回 `n` 的最大质因子，要求 `n >= 2`。
- 当前 `solve()` 对应常见题型：若 `n` 是质数则输出 `Prime`，否则输出最大质因子。
- 使用 GCC/Clang 的 `__uint128_t` 完成无溢出模乘，编译标准为 C++17。

完整代码

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using poly = vector<ll>;
using u64 = uint64_t;

u64 mul_mod(u64 a, u64 b, u64 mod) {
    return static_cast<u64>(static_cast<u128>(a) * b % mod);
}

u64 pow_mod(u64 a, u64 b, u64 mod) {
    u64 res = 1 % mod;
    while (b) {
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                  19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
```

```
    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }

    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL,
                  450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;

        u64 x = pow_mod(a % n, d, n);
        if (x == 1 || x == n - 1) continue;

        bool passed = false;
        for (int r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) {
                passed = true;
                break;
            }
        }
        if (!passed) return false;
    }
    return true;
}

mt19937_64 rho_rng(
    chrono::steady_clock::now().time_since_epoch().count());

u64 add_mod(u64 a, u64 b, u64 mod) {
    return a >= mod - b ? a - (mod - b) : a + b;
}

u64 pollard_rho(u64 n) {
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                  19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return p;
    }

    constexpr u64 BATCH = 128;
    constexpr u64 ROUND_LIMIT = 1ULL << 20;
    uniform_int_distribution<u64> dist(1, n - 1);

    while (true) {
        const u64 c = dist(rho_rng);
        u64 y = dist(rho_rng);
        u64 x = 0, ys = 0;
        u64 g = 1, q = 1, len = 1;

        auto f = [&](u64 v) {
            return add_mod(mul_mod(v, v, n), c, n);
```

```
        };

        while (g == 1 && len <= ROUND_LIMIT) {
            x = y;
            for (u64 i = 0; i < len; ++i) y = f(y);

            for (u64 k = 0; k < len && g == 1; k += BATCH) {
                ys = y;
                const u64 step = min(BATCH, len - k);
                for (u64 i = 0; i < step; ++i) {
                    y = f(y);
                    const u64 diff = x > y ? x - y : y - x;
                    q = mul_mod(q, diff, n);
                }
                g = gcd(q, n);
            }
            len <<= 1;
        }

        if (g == 1) continue;
        if (g == n) {
            do {
                ys = f(ys);
                const u64 diff = x > ys ? x - ys : ys - x;
                g = gcd(diff, n);
            } while (g == 1);
        }
        if (g != n) return g;
    }
}

void factor_max(u64 n, u64 &ans) {
    if (n <= ans) return;
    if (is_prime(n)) {
        ans = n;
        return;
    }

    u64 d = pollard_rho(n);
    u64 other = n / d;
    if (d < other) swap(d, other);
    factor_max(d, ans);
    factor_max(other, ans);
}

u64 max_prime_factor(u64 n) {
    u64 ans = 0;
    factor_max(n, ans);
    return ans;
}

void solve() {
```

```
    u64 n;
    cin >> n;

    if (is_prime(n)) {
        cout << "Prime\n";
    } else {
        cout << max_prime_factor(n) << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    cin >> T;
    while (T--) solve();
    return 0;
}
```

设计说明

原实现使用 Montgomery 模乘并夹带大量与本题无关的宏和输入输出封装，迁移到当前火车头后会产生重复类型、重复函数和不一致的调用方式。这里直接使用火车头已有的 `u128` 做模乘，代码更短，且能覆盖完整的 `u64` 范围。

Pollard-Rho 使用 Brent 判圈，并将一批差值的乘积合并后再求一次 `gcd`，比逐步求 `gcd` 的 Floyd 写法常数更小。分解时先递归较大的因子，并用当前答案剪枝；因为目标只是最大质因子，所以无需保存并排序全部质因子。

`is_prime` 最坏需要常数个底数的快速幂，每次快速幂为 `O(log n)` 次模乘。Pollard-Rho 是随机算法，分解合数的期望复杂度约为 `O(n^(1/4))`，实际处理 `1e18` 量级整数通常很快。

接入现有火车头

现有 `火车头.cpp` 已经定义了 `u64`、`u128`、`mr_mul`、`mr_pw` 和 `is_prime`。若不是直接使用上面的完整代码，而是向火车头中追加模块：

1. 保留火车头的类型别名。
2. 用本文的 `mul_mod`、`pow_mod`、`is_prime` 替换火车头中对应的 Miller-Rabin 三个函数，避免重定义。
3. 将 `rho_rng` 至 `max_prime_factor` 这一段放在 `solve()` 前。
4. 用本文的 `solve()` 替换空的 `solve()`；`main()` 可继续使用火车头原版。

如果题目要求输出完整质因数分解，而不是最大质因子，应去掉 `n <= ans` 剪枝，改为把素因子存入 `vector<u64>`，最后排序并统计指数。